

Lab -6:

Experiment No: 6

Aim

To create, compile, deploy, and perform transactions on a Ballot smart contract using Solidity and Remix IDE by changing the proposal name from bytes32 to string.

Theory

A smart contract is a program stored on a blockchain that automatically executes according to predefined rules. Solidity is a programming language used to develop Ethereum smart contracts, while Remix IDE provides an environment to write, compile, deploy, and test smart contracts.

The Ballot smart contract implements a voting system in which voters can be given voting rights and can vote for different proposals.

Important Solidity Concepts

1. **require**
Used to check whether a specified condition is satisfied. If the condition is false, the transaction is reverted.
2. **mapping**
Used to store information associated with Ethereum addresses. In this experiment, the voters mapping stores voter information.
3. **storage**
Stores data permanently on the blockchain.
4. **memory**
Provides temporary storage during function execution.
5. **bytes32**
A fixed-size data type that can store up to 32 bytes and is efficient for short names.
6. **string**
Stores variable-length text and allows readable proposal names.

Procedure

Step 1: Open Remix IDE

Open **Remix IDE** in a web browser.

Step 2: Open the Ballot Contract

1. Open the **File Explorer**.

2. Open the **contracts** folder.
3. Open the **3_Ballot.sol** contract.

Step 3: Modify the Proposal Structure

In the Proposal structure, change:

bytes32 name;

to:

string name;

```
// Represents a single proposal
struct Proposal {
    string name;           //
    uint voteCount;       //
}
```

Step 4: Modify the Constructor

Change:

constructor(bytes32[] memory proposalNames)

to:

constructor(string[] memory proposalNames)

```
// khushi shukla_29
constructor(string[] memory proposalNames)
    chairperson = msg.sender;
```

Step 5: Modify winnerName()

Change the return type of winnerName() from:

bytes32

to:

string memory

These modifications allow proposal names such as "Khushi Shukla", "Bob", and "Charlie" to be stored as strings.

```
function winnerName() external view returns (string memory winnerName_)
{
    winnerName_ = proposals[winningProposal()].name;
}
```

Step 6: Compile the Contract

Open **Solidity Compiler**.

Select a compatible Solidity compiler version.

Select 3_Ballot.sol.

Click **Compile 3_Ballot.sol**.

Verify that the contract compiles successfully.

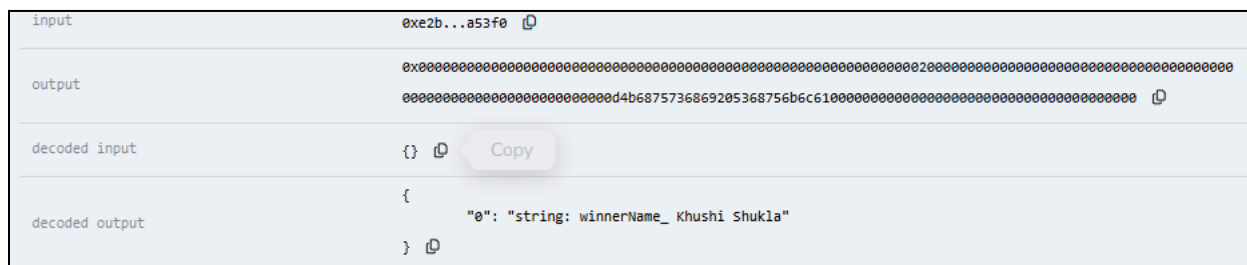
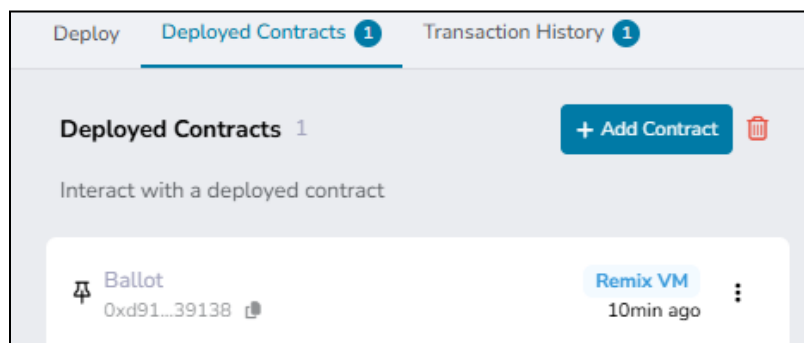
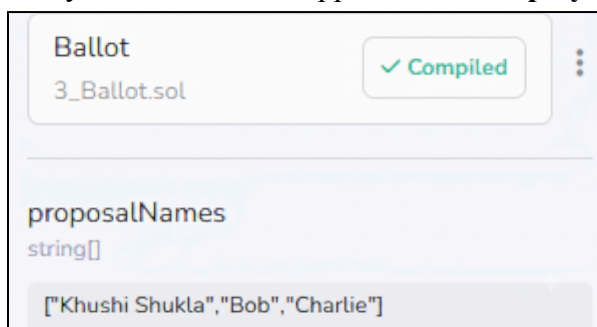


Step 7: Deploy the Contract

1. Open **Deploy & Run Transactions**.
2. Select **Remix VM** as the environment.
3. Select the **Ballot** contract.
4. In the constructor input, enter:

["Khushi Shukla","Bob","Charlie"]

5. Click **Deploy**.
6. Verify that the contract appears under **Deployed Contracts**.



Step 8: View Proposal Details

Under **Deployed Contracts**, use the proposals function.

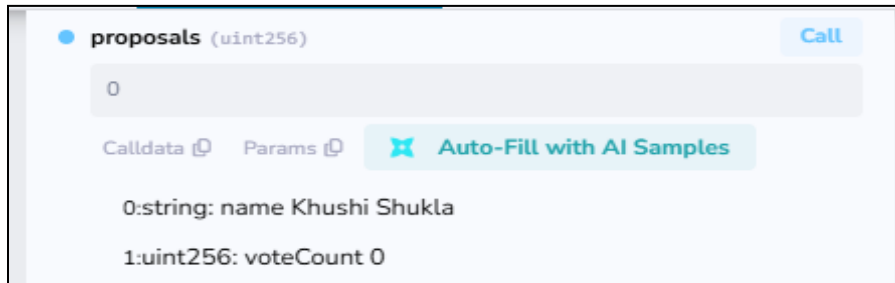
Enter the following indices one at a time:

0 to view **Khushi Shukla**,

1 to view **Bob**, and

2 to view **Charlie**.

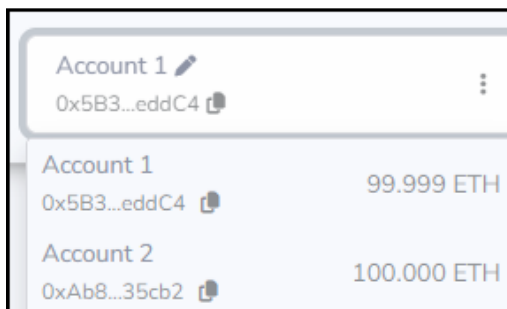
This verifies that the proposals were successfully stored in the contract.



Step 9: Give Account 2 the Right to Vote

1. Copy the address of **Account 2** from the Account list.
2. Switch to **Account 1**, which is the chairperson.
3. In the deployed Ballot contract, open giveRightToVote.
4. Enter the **Account 2 address**.
5. Click **transact**.

The chairperson grants Account 2 the right to vote.



Step 10: Cast the Vote

1. Switch the selected account to **Account 2**.
2. Open the vote function.
3. Enter: 0
4. Click **transact**.

Account 2 therefore votes for **Proposal 0 — Khushi Shukla**

giveRightToVote address

0xAb8483F64d9C6d1EcF9b849Ae677dD3:

Calldata Parameters

Value 0 wei

Gas limit auto 0

Transact

Step 11: Verify the Vote

1. Open the voters function.
2. Enter the **Account 2 address**.
3. Click **call**.
4. Verify that the voter information shows that the account has voted.

vote uint256

0

Calldata Parameters

Value 0 wei

Gas limit auto 0

voters address

0xAb8483F64d9C6d1EcF9b849Ae677dD3:

Calldata Parameters

0:uint256: weight 1

1:bool: voted true

2:address: delegate 0x00000000000000000000000000000000

3:uint256: vote 0

This confirms that the vote has been successfully recorded.

Result

The Ballot smart contract was successfully modified, compiled, deployed, and tested using Solidity and Remix IDE. The proposal name was changed from bytes32 to string, and the contract successfully demonstrated proposal creation, granting voting rights to another account, and casting and verifying a vote.

Conclusion

The experiment provided practical understanding of smart contract deployment and transactions, along with the use of require, mapping, storage, and memory in Solidity. It also demonstrated the difference between bytes32 and string for storing proposal names.